

Using the Active Bill Of Materials

coreBuilder Application Note

Ronnie Craig, Synopsys Design Reuse Group

August 2000

Abstract

It is the responsibility of the core developer to include all necessary data and information with a packaged core, so that the end user may make the most effective use of that core. The Bill Of Materials (BOM) specifies the full set of core deliverables.

This Application Note looks at how the Bill Of Materials may be customized through the use of a template, to meet differing deliverable requirements. It also looks at the Active BOM features in detail.

Introduction

The BOM Template is a file that specifies the rules that define a standard for IP deliverables. The coreBuilder tool is shipped with a template that specifies the minimum set of deliverables specified in the second edition of the RMM, and customers will be allowed to specify their own Template files.

When a developer creates a new coreBuilder workspace, they will specify a BOM template to be used for the IP. coreBuilder will read the template and load it into the workspace. This will define the deliverables needed for the core, and the default rules for loading them relative to the designer's workspace.

Within the Bill Of Materials, deliverable files are grouped into categories (filegroups) according to their type and purpose. The Active BOM functionality allows the developer to browse the filegroups, and manually add files to them. The facility allows the developer to make some of the deliverables configurable, specify configuration dependencies between the deliverables, and preview the configuration dialog for the configurable deliverables. In addition, the core developer can check the completeness and conformance of their Bill of Materials during the BOM creation activity.

Figure 1 : Overall data flow shows the high-level data flow for the project. The BOM Template is an input to coreBuilder when the initial coreBuilder workspace is created.

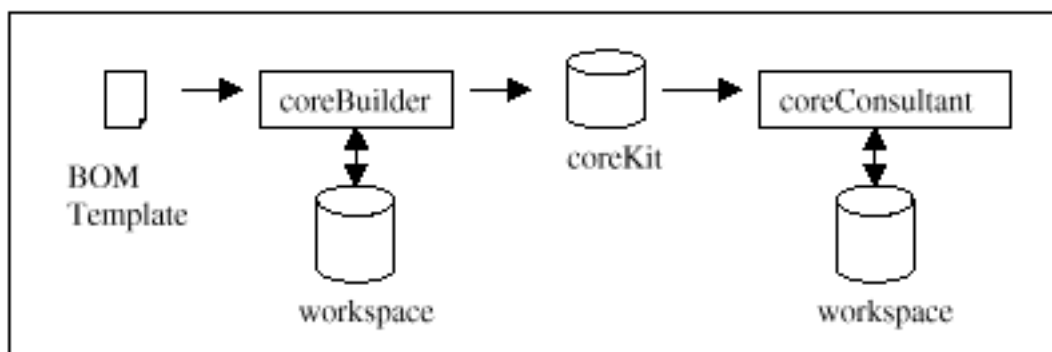


Figure 1 Overall data flow

BOM Template File

When a designer creates a new coreBuilder workspace they will specify the BOM template to be used for the core. coreBuilder ships with a template for the soft IP deliverables from the RMM, and the site or user can also specify their own template files.

The BOM template is a file of Tcl commands. The file will be loaded into the workspace kb, and will be re-executed each time the workspace is loaded, so it is acceptable to put both the template definition and procedures it refers to into the template file. It also means that the template can be manipulated with Tcl commands to build up the template. Therefore it is possible to structure a template which shares some base set of deliverables with other templates, and then appends deliverables to the base.

The template for the deliverables is specified through a Tcl variable that contains a series of nested Tcl lists. Each of the fields contains a keyword and zero or more additional strings. Each Keyword specifies either an attribute or defines a new object, and both are defined in the containing context.

This section will explain the construction, and key elements of, the BOM template.

1. Standard Header

First of all, each BOM template has an identical header -

```
set BOMTemplate {  
    BOMTemplate  
    <name>  
    <version information>  
    <Groups/Deliverables>  
}
```

2. Template Name

Next, it is necessary to make this template unique by giving it a name -

```
set BOMTemplate {  
    BOMTemplate  
    {Name {My BOM Template}}  
    <version information>  
    <Groups/Deliverables>  
}
```

The option 'My BOM Template' will now appear in the pull-down list of BOM templates, when the core developer creates a new workspace in coreBuilder.

3. Version Information

The version information lines describe the version of the coreBuilder format for the BOM Template, and the version of this BOM Template -

```
set BOMTemplate {
  BOMTemplate
  {Name {My BOM Template}}
  {coreBuilderBOMTemplateVersion 1.0}
  {Version 1.0}
  <Groups/Deliverables>
}
```

4. BOM Groups and Deliverables

BOM deliverables are typically placed in logical groups. For example, all types of documentation may be placed in a 'Documents' group, and so on. The example below shows an example of a BOM group containing two simple deliverables ('Functional Specification' and 'Datasheet') -

```
set BOMTemplate {
  BOMTemplate
  {Name {My BOM Template}}
  {coreBuilderBOMTemplateVersion 1.0}
  {Version 1.0}
  {Group
    {Importance Required}
    {Name "Documentation Files"}
    {Deliverable
      {Name "Functional Specification"}
    }
    {Deliverable
      {Name "Datasheet"}
    }
  }
}
```

First of all, the 'Group' keyword is used to denote the start of a group in the template. The first line after that contains the 'Importance' keyword. This indicates how critical these deliverables are in the group (see *Appendix 1 : BOM Template Attributes* for a full description). In this example, we require these particular deliverables to be present in the Bill Of Materials.

After this, the name of the deliverable group is specified. This will appear in the Bill Of Materials filegroup spreadsheet during the 'Create Bill Of Materials' activity in coreBuilder. In this example, the Group is called "Documentation Files", and contains two deliverables - "Functional Specification" and "Datasheet".

To make your BOM Template available when creating a new workspace in coreBuilder, you need to add the directory containing the BOM Template file to the UNIX variable `rt_bom_template_dirs` -

```
%set rt_bom_template_dirs = ( $rt_bom_template_dirs \  
                             <my_bom_template_dir> )
```

When creating a new workspace in coreBuilder, you can then specify your own BOM template -

```
coreBuilder>create_workspace -bom_template \  
                             <my_bom_template_name> \  
                             -name <workspace_name>
```

Or you can pick a BOM template from the drop-down list in the GUI.

Populating the Bill Of Materials

Before the final creation of a coreKit, it is necessary to populate the BOM. The BOM contents are as specified by the BOM template described in the previous section.

Certain key elements of the BOM are automatically populated, and the remainder must be completed by the core developer.

The ‘Synopsys’ Deliverable Group

The ‘Synopsys’ deliverable group is in the BOM by default. This group contains three sets of deliverables -

- Synopsys/Hdl
All HDL source code associated with the core.
- Synopsys/DesignKbs
Knowledge Bases associated with each design in the core hierarchy. These files are stored in the <workspace_name>/kb directory when the developer has created a workspace, and are copied into the BOM.
- Synopsys/Plugin
coreBuilder customization kbs stored in the directory specified during the ‘LoadConsultantPlugins’ activity.

[Note : The ‘Synopsys’ filegroup cannot be modified]

User-defined Deliverable Groups

In addition to the Synopsys default deliverable group, other groups may be defined in the BOM Template (see the *4. BOM Groups and Deliverables* example on page 6).

Alternatively, if more file groups are required, in addition to those specified in your BOM template, they may be added in batch or GUI mode.

Adding File Groups - Batch

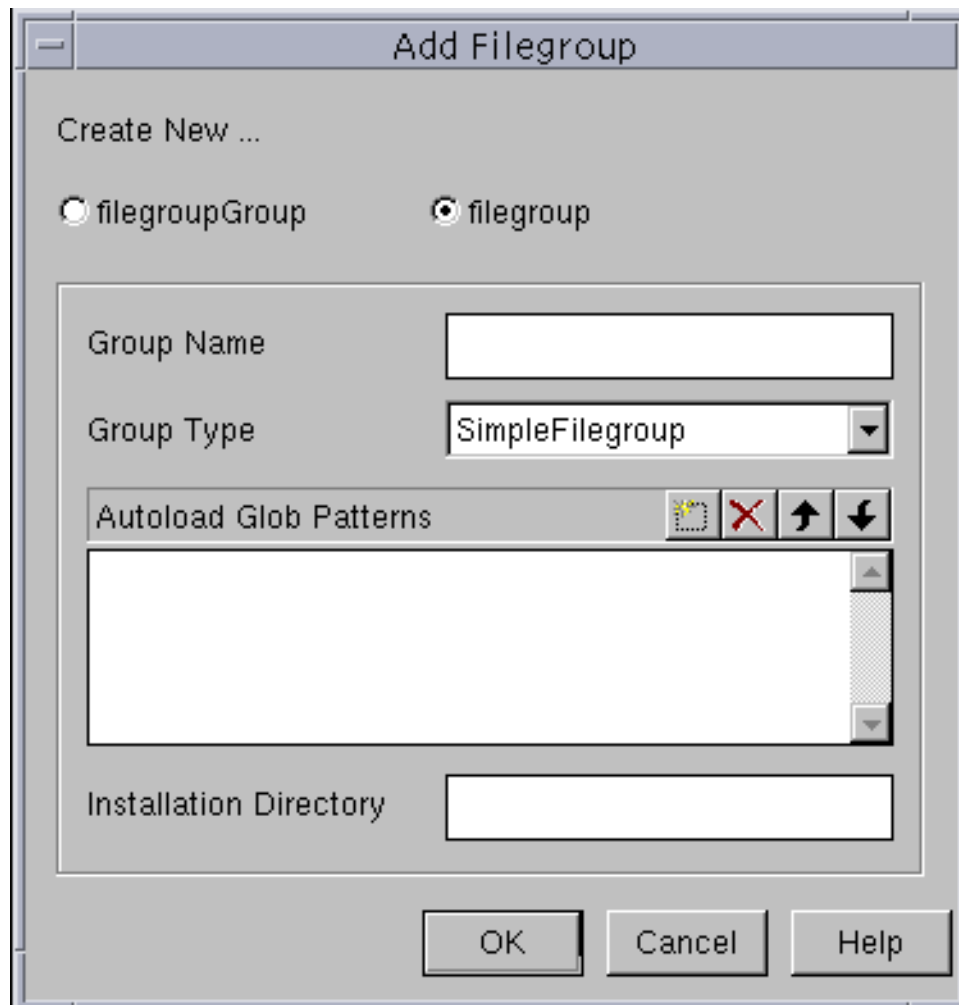
The command 'create_autoload_filegroup' can be used in batch mode to create a new file group in the BOM.

```
coreBuilder> create_autoload_filegroup -patterns doc/*  
               -install doc Documentation
```

This command will create a new filegroup called 'Documentation', and populate it with files from the 'doc' directory in the developers working area. When installed, a 'doc' directory will be created in the installation area to contain the documentation files.

Adding File Groups - GUI

In the 'Create Bill Of Materials' dialog, click on 'Add' in the 'Filegroup Editing' menu group. This will launch the dialog shown in below -



If you choose to create a new file group, select 'filegroup'. Enter the name of the filegroup in the Group Name box (this will subsequently appear in the BOM spreadsheet).

There are two ways of populating file groups -

1. SimpleFileGroup

If this option is chosen, simply add the files manually using the 'Autoload Glob Patterns' file browser. glob patterns may also be used here if required (see example below).

2. AutoloadFileGroup

For filegroups containing a number of files, it may be preferable to use some 'glob patterns' (wildcard-type file selection patterns) to specify the files in the file group.

Choose the 'AutoloadFileGroup' Group Type in the 'Add File' dialog, and specify the Group Name. Then, in the 'Autoload Glob Patterns' window, specify the patterns either as <directory_name>/<glob_pattern> or simply <glob_pattern>.

For example -

sim/* - will load all files from the 'sim' directory

Add files to File Group - Batch

Use the 'add_files_to_filegroup' command to add files to each deliverable filegroup, as shown in the example below -

```
coreBuilder>add_files_to_filegroup Documentation \  
             -files "doc/*.pdf" \  
             -install Documentation
```

The command above will add the file risc8.pdf, found in the directory 'doc', to the filegroup Documentation. When the coreKit is installed by the user, a 'Documentation' directory will be created which will contain the risc8.pdf file.

[Note: The 'add_files_to_filegroup' command (see next section) will also create a filegroup if one doesn't already exist with the specified name]

Add files to File Group - GUI

In GUI mode, files can be added to the file groups by selecting the appropriate filegroup from the 'Filegroups' in the 'Create Bill Of Materials' dialog. Then click 'Files' then 'Add File' to bring up the dialog shown in *Figure 2 Add files to BOM File Group Dialog* below -

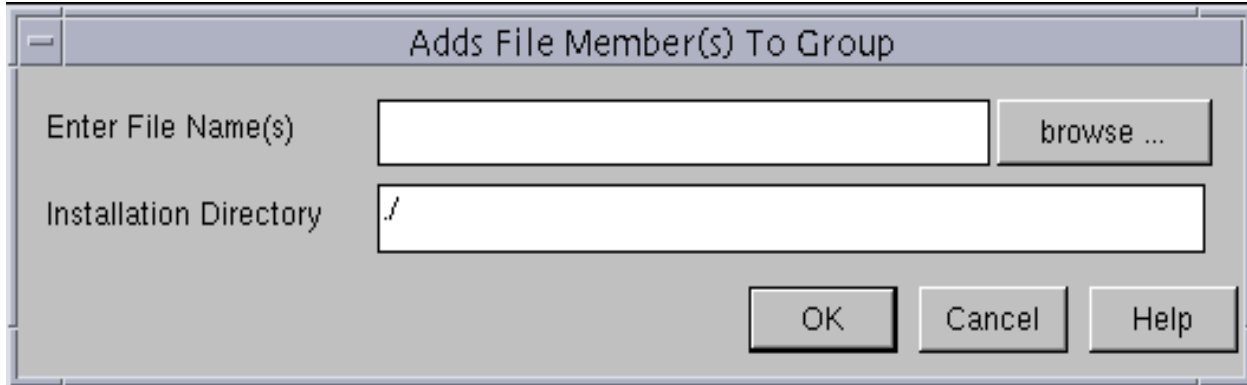


Figure 2 Add files to BOM File Group Dialog

Select the file to be added by clicking 'Browse'. To specify an installation directory other than the root coreKit installation directory, add the installation directory name in the 'Installation Directory' box.

Customizing Files and File Groups

A key feature of the Active BOM is the fact that deliverables can be made customizable by the developer. Configurable filegroups are groups of files that can be configured based on the settings of one or more parameters. When filegroups are configurable, there need to be activities in the coreConsultant flow which guide the user through the specification of the parameter values, and then configures and installs the files in the filegroup. The activity that configures the

filegroup may be specific to the given filegroup, or the filegroup may reference other parameters that are configured in some other built-in activity (such as the RTL source configuration).

The configuration of the files in a filegroup can take two forms: selective installation and file configuration using reuse-pragma substitution. Selective installation is when different deliverables are written to the integrator workspace based on the parameters specified. For example, writing out a different testbench for different parameter settings of the RTL code.

Pragma substitution preprocesses the files in the filegroup, replacing text in reuse-pragma blocks with the result of some function which depends on configuration parameters, and the preprocessed files get installed into the users workspace. For further information about text substitution, please refer to Chapter 4 of the coreBuilder User Guide, specifically the section on text substitution.

This configuration functionality is broken into several pieces:

- Specifying the configuration intent for a group of files.
- Providing general reuse-pragma substitution procedures for use in non-hdl files.
- Analyzing the files to process their intent and allowing the user to view/change the configuration intent for the filegroup.
- Allowing the user to preview the dialog that will be used by the coreConsultant for configuration of the filegroup.
- Setting up a relationship to an activity in the coreConsultant flow which will prompt the user for the parameters, and then configure and write the filegroup. This may be a built-in activity or an activity may need to be generated for the express purpose of configuring the filegroup.

In addition, certain filegroups are configured automatically by a pre-existing activity (or customization) in the coreConsultant flow.

Configuration Intent

Configuration intent is specified in two places. A Tcl command file can be specified to define the parameters used to configure the filegroup. The user may also annotate the text of the files with a set of reuse-pragmas, which define the substitutions that will be made to the source.

For files other than the RTL source, a very simple analyzer is provided. This analyzer will only check to see if a file needs to be configured by marking ASCII files that contain the word reuse-pragma as needing to be substituted when they are written. All parameter definition will be done using intent scripts.

The procedures provided to perform reuse-pragma based substitution in HDL files are defined in the coreBuilder application note on HDL annotations. In non-HDL files, it is recommended that the ReplaceFormatParam and parameter_conditional_text commands are used.

- `ReplaceFormatParam`

Used to replace a region in a text file with the results of the ReplaceFormatParam command. For further information please refer to the ReplaceFormatParam man page.

- `parameter_conditional_text`

Similarly to ReplaceFormatParam, parameter_conditional_text will allow conditional inclusion of text in a text file. For further information please refer to the parameter_conditional_text man page.

The designer controls how the configuration information is loaded by specifying attributes on the filegroup that is configurable. The set of attributes is explained in the table below:

Table 1 Configurable Filegroup Attributes

filegroup attributes for supporting configuration				
Attribute Name	Type	r/w	Default	Description
Install	boolean	r/w	true	Specifies if the filegroup should be installed. To support conditional installation, the value can be set to a formula that uses eval_param to reference parameter settings in calculating if the filegroup should be installed.
ConfigIntentSearchPath	list of directories	r/w		Specifies the search path used for identifying files containing configuration intent for the files in this filegroup. By default, configuration intent will be searched for in the same directory the file was loaded from.
Label	string	r	filegroup name	Specifies the label used for this filegroup in the GUI dialog.
Parameters	Item list	r	""	A collection of the parameters defined by this filegroup for its configuration.
Configurable	boolean	r/w	false	Specifies that this file group is configurable

filegroup attributes for supporting configuration				
Attribute Name	Type	r/w	Default	Description
ConfigDependsOnGroup	list of file groups	r/w	""	This is a list of the filegroups that must be configured before this filegroup is configured.
ConfigDependsOnActivities	list of activities	r/w	""	This is a list of the activities that must be completed before this filegroup is configured.
PostConfigCmd	Tcl command	r/w	""	A Tcl command that will be executed after the filegroup is configured and written out.
ConfigActivity	""	r/w	""	If the filegroup does not have parameters then this string is used to specify the activity whose completion will cause this filegroup to be installed. If "" is specified then the filegroup will be installed when the coreKit is installed. If no activity can be found with the specified name one will be automatically added to the coreConsultant flow. For filegroups with their own parameters this attribute specifies the name of the activity which will be created (if it doesn't already exist) for configuring and writing out the filegroup.

filegroup attributes for supporting configuration				
Attribute Name	Type	r/w	Default	Description
ActivityGroup	string	r/w	""	Activity group for any activity that is automatically created for this filegroup.
ActivitySequence	integer	r/w	0	Sequence number for any activity that is automatically created for this filegroup.

Once the attributes are set on the filegroup, the user will select the filegroup and then specify that the filegroup is configurable. This will analyze the files and create the configuration parameters. Once that is complete the user will be able to view/modify the configuration intent and preview the configuration dialog for the filegroup.

Example Filegroup Configuration Setup

The example below shows a typical configuration intent file. The file contains the commands to define a parameter, which is used to configure a set of files. Additional parameters can be specified as needed.

The following is an example of a set of Tcl commands to setup a configurable file group.

```
set doc_fg [create_autoload_filegroup \
                Documentation/ChooseFormat \
                -patterns {doc/*} \
                -install Documentation]
set_filegroup_attribute $doc_fg FilePerms 0755
set_filegroup_attribute $doc_fg Configurable true
```

```
set_filegroup_attribute $doc_fg Label "Choose Format"  
set_filegroup_attribute $doc_fg ConfigActivity OpenDoc  
set_filegroup_attribute $doc_fg ConfigIntentSearchPath doc  
set_filegroup_attribute $doc_fg PostPromptCmd opendoc  
set_filegroup_attribute $doc_fg ActivityGroup "Open  
Documentation"
```

This example creates an autoload filegroup called Documentation/ChooseFormat in the BOM. This file group contains all the files in the developers 'doc' directory, and on installation will create a directory called 'Documentation' in the installation area to store all of these documentation-related files.

Following creation of the filegroup, the `set_filegroup_attribute` command is used to set attributes on this filegroup. See *Table 1 Configurable Filegroup Attributes* for more details about these attributes. In *Figure 3 Example Activity Dialog* below, the Activity Group can be seen as a tab in coreConsultant.

The next step is to define what will happen when the user selects the 'Choose Format' button. This is achieved by setting up some configuration parameters for this filegroup. The configuration parameters are stored in a separate Tcl intent script in the specified search path (defined by the `ConfigIntentSearchPath` attribute in the example above).

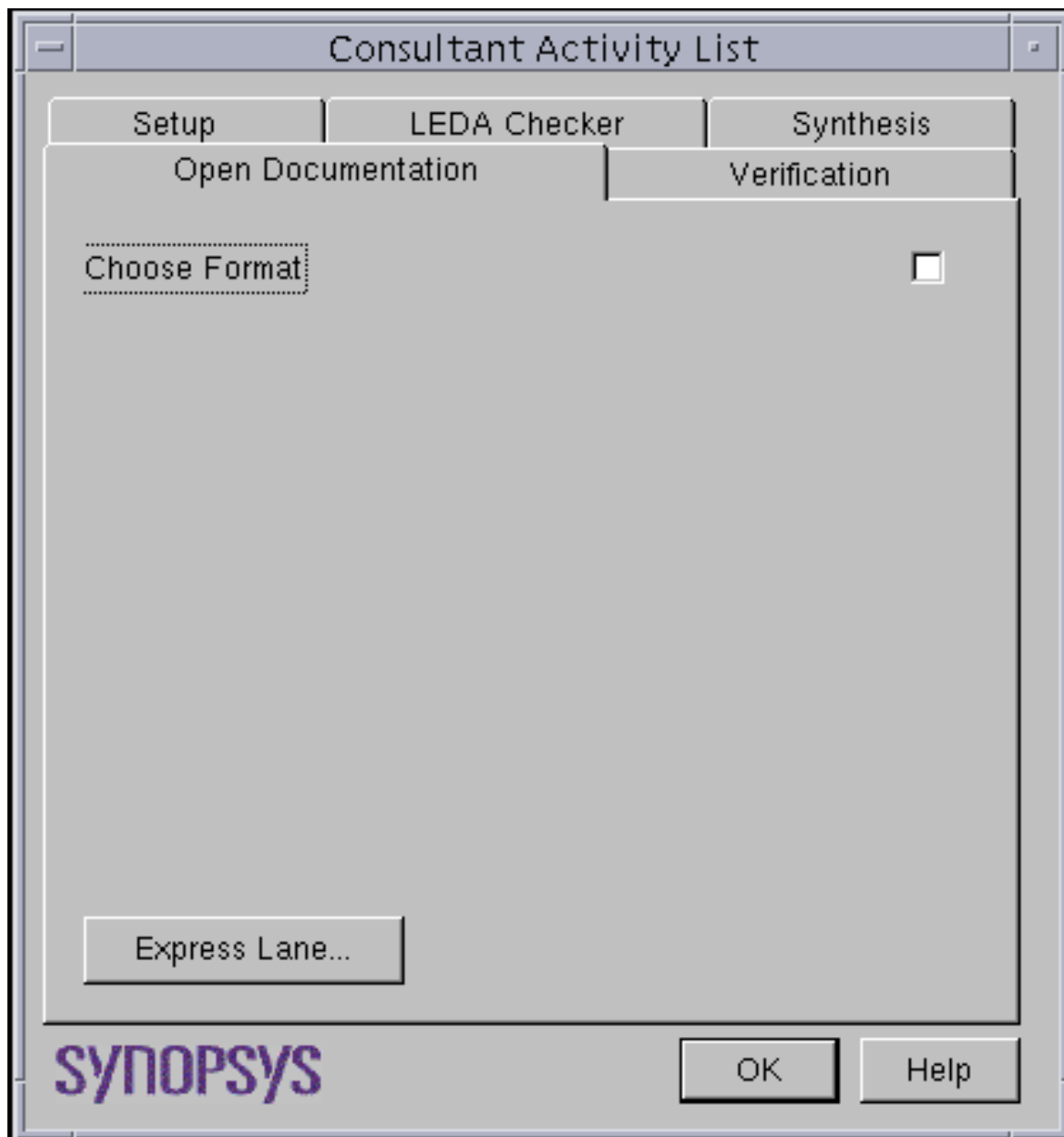


Figure 3 Example Activity Dialog

Example Filegroup Configuration Intent

The example below shows the simple Tcl commands required to add some configuration information to a file group.

```
set d [create_configuration_parameter \  
    -type boolean \  
    -label "Open Documentation" \  
    openFile]  
set_attribute $d -attr SymbolicNames -value "PDF HTML"
```

When the user selects 'Load' from the custom activity dialog, a new configuration dialog will popup as shown below.

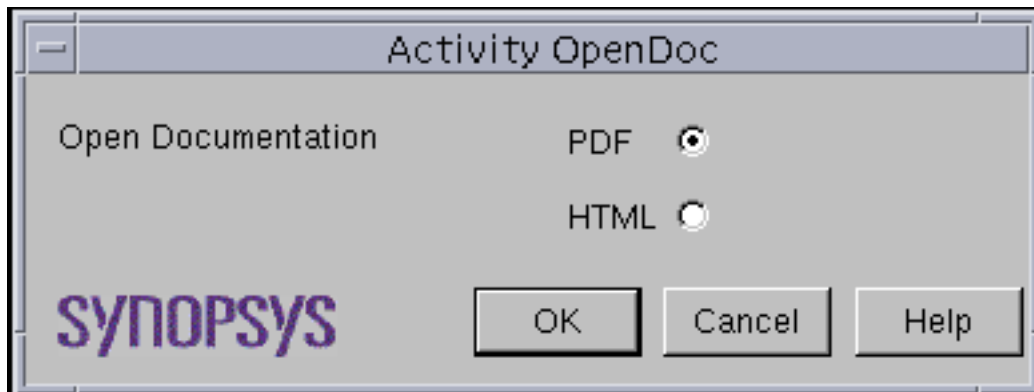


Figure 4 Custom Activity Configuration Dialog in coreConsultant

The relationship between the sample Tcl configuration and the configuration dialog can be clearly seen.

Once the configuration has been selected (i.e. PDF or HTML) by making a selection and clicking OK, the 'PostPromptCmd' command is executed. The PostPromptCmd attribute in this case will be a Tcl proc which decides which file viewer to launch based on the file type chosen by the user. The proc name in this example is 'opendoc', and is effectively included as a customization plugin.

For more information about constructing and implementing customization plugins please refer to the Application Note entitled 'Customizing the coreBuilder and coreConsultant Activity Flow'.

Appendix 1 : BOM Template Attributes

This Appendix contains full details of all attributes which are applicable to BOM Templates.

Keyword	Type	Description
Name	<string>	Name of the group of deliverables
Label	<string>	Label to be shown to the user for this group
Importance	Enum {Required, Optional, OneRequired}	Indicates the default criticality of including the deliverables in the group. The primary values are Required and optional. OneRequired specifies that at least one deliverable in the OneRequiredGroup may be specified. The default value for the attribute is Optional.

Table A1 BOM Group Attributes

Keyword	Type	Description
Name	<string>	Name of the deliverable
Label	<string>	Label to be shown to the user for this deliverable
Importance	Enum {Required, Optional, OneRequired}	Indicates the criticality of including the deliverable. The primary values are Required and optional. OneRequired specifies that at least one deliverable in the OneRequiredGroup may be specified. The default value for the attribute is Optional.
OneRequiredGroup	<string>	This string specifies that either this deliverable, or another deliverable in the group with the same value of OneRequiredGroup is required to be specified.

Keyword	Type	Description
DeliverableType	Enum {RTL, TRE, Other}	This specifies the type of information that is stored in the deliverable. It is the way the BOM Template can tell coreBuilder where the file types processed by coreBuilder belong in the deliverables. There will be a pre-defined set of types that coreBuilder understands, but these types can be extended, by user-defined types too.
DefaultLoadPath	List of <glob string>	A Tcl list of glob-style patterns specifying the path to load this deliverable from. Relative paths are assumed to start at the root of the workspace for the core. This is just a default.
DefaultInstallDir	<path>	Installation directory for the deliverable relative to the root directory of the coreConsultant workspace.
Exists	Boolean	This value will be copied to the Exists attribute on the coreBuilder filegroup for this deliverable. By default the value is set to a formula that checks for files being found using the DefaultLoadPath.
IsUpToDate	Boolean	This value will be copied to the IsUpToDate attribute of the coreBuilder filegroup for this deliverable. This attribute specifies if the files loaded into the coreKit are up-to-date with respect to the files on disk. The default for this attribute is TRUE, because in the default case the files will not actually be loaded until the installation coreKit is built.

Table A2 BOM Deliverable Attributes

